

66-146 Modeling Language Evolution

Julius W, Liam G, Andrew S, Leila T

Introduction

This is the final report for the 66-146 group project on modeling language evolution. Our project encompasses the development of a Markov Chain model to simulate the mutation and evolution of languages over time. This would allow us to study trends and characteristics of language evolution in a controlled environment and perhaps even extend it to real world applications.

Language develops to accommodate one's immediate community of fellow speakers, and as such becomes a part of the identity of that community. Communities are built on cultural practices and geographical location. Over time, small changes in the environment and social settings will lead to long-term divergence in language use. These small differences in the use of language form the basis for dialectal formation. When two languages come into contact, they will begin to adopt the features of each other. When there are two languages that interact with each other and one has more power, the less powerful will slowly dwindle and possibly die out. Over time, language becomes optimized to best serve the needs of its speakers. Children speak a different version of the language from their parents as they put together various grammatical cues from their environment in slightly different ways.

In this report, we will outline the problem we attempt to model, the mathematical frameworks and foundations that we will utilize in building our model, the specific components and rules of our model, and finally we will conclude with our model analysis and interpretation of results.

Mathematical Framework of the Model

We will first define our universe over $U = \{L_1, L_2, \dots, L_n\}$ where each L_i represents a distinct language. **For example**, L_1 could represent **Proto-Indo-European**, L_2 could represent a derived variant such as **Latin**, and so on. Each language L_i becomes a further variant of the previous language L_{i-1} and our base language L_1 is the root of all languages in our universe U .

Since we are modelling the mutation of one language, we cannot have that L_1 is able to mutate to L_3 , for example without first mutating to L_2 . Thus, we can define our transition matrix P as follows:

$$P = \begin{bmatrix} p_{11} & p_{12} & 0 & 0 & \cdots & 0 \\ 0 & p_{22} & p_{23} & 0 & \cdots & 0 \\ 0 & 0 & p_{33} & p_{34} & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & p_{nn} \end{bmatrix}$$

Where p_{ii} represents the probability of language L_i remaining and $p_{i,j}$ represents the probability of language L_i mutating to L_j . We first do not define the exact probabilities of mutations as these will become variables in our users interface to allow for flexibility in the model.

Defining Our Agent Space

We will define our space in 2D by a set

$$S = \{(x, y) \in \mathbb{N}^2 \mid (0 \leq x < W) \wedge (0 \leq y < H)\}$$

where each $(x, y) \in S$ represents a coordinate in our 2D space. Each coordinate is a place where an agent is able to be positioned at during any given tick.

We will define our agents as a set $A = \{a_1, a_2, \dots, a_m\}$ where each agent a_i has a state defined by a language $L_j \in U$ and a position defined by a coordinate such that

$$a_i = (L_j, (x, y)) \text{ where } L_j \in U \text{ and } (x, y) \in S \text{ for } a_i \in A$$

We will define $L(a_i)$ to be the language state of agent a_i and $P(a_i)$ to be the position of agent a_i .

Modelling Movement Over Our Time Interval

We will define our time interval as discrete ticks $T = \{0, 1, 2, \dots, t\}$ where each tick represents a given tick in our simulation. At each tick, each agent $a_i \in A$ will have a chance to move to a new position in S based on the states of surrounding agents.

For this we can first define the set of **interactable neighbors**. For an agent a_i with

state $(L(a_i), P(a_i))$ at tick t_k , we can define the set of interactable neighbors as a subset $A_n \subseteq A$ where each agent $a_j \in A_n$ is a distance r away from a_i and the probability of language mutation between $L(a_i)$ and $L(a_j)$ is greater than some threshold value p_{input} .

Interactable Neighbors. For an agent a_i with state $(L(a_i), P(a_i))$ at tick t_k , we define the set of **interactable neighbors** as a subset $A_n \subseteq A$ where each $a_j \in A_n$ satisfies

$$d(P(a_i), P(a_j)) \leq r \quad \text{and} \quad p_{L(a_i), L(a_j)} > p_{input}.$$

Here, d is the distance between the positions of the agents, r is the maximum interaction radius, and p_{input} is a user-defined threshold for the minimum mutation probability required for interaction.

Movement Rule. Now that we have a set $A_n \subseteq A$ of interactable neighbors for agent a_i at tick t_k , we can define the movement rule by a function Φ that takes an input of the positions of all agents $a_k \in A_n$ and outputs a new position for our target agent a_i at tick t_{k+1} .

We can define Φ as a mapping $\Phi : \{P(a_k) \mid a_k \in A_n\} \rightarrow (x, y) \in S$. First I want to note that an agent can only move to a position that is only one unit away from its position at t_k and also to a position not occupied by another agent. Thus, we can define the possible movement positions for agent a_i at tick t_k as

$$s(U) = (0, 1), \quad s(D) = (0, -1), \quad s(L) = (-1, 0), \quad s(R) = (1, 0), \quad s(S) = (0, 0)$$

From this we can now actually slightly modify our mapping Φ to be a mapping $\Phi : \{P(a_k) \mid a_k \in A_n\} \rightarrow s(M)$ where $M = \{U, D, L, R, S\}$.

Modelling Interaction and Mutation

We can model the interaction and mutation of languages between agents based on their interactable neighbors. Since we have defined A_n to be the set of interactable neighbours for a given agent a_i at tick t_k , we can define the mutation process as follows:

Interaction and mutation. Fix a radius $r > 0$ and (optionally) a mutation threshold $p_{in} > 0$. For agent a_i at tick t_k with state $(L_{t_k}(a_i), P_{t_k}(a_i))$, define the set of interactable neighbors

$$A_n(i, t_k) = \{a_j \in A \setminus \{a_i\} \mid d(P_{t_k}(a_i), P_{t_k}(a_j)) \leq r, p_{L_{t_k}(a_i), L_{t_k}(a_j)} > p_{in}\}.$$

If $A_n(i, t_k) = \emptyset$, then no interaction occurs and the language remains unchanged:

$$L_{t_{k+1}}(a_i) = L_{t_k}(a_i).$$

Otherwise, choose one partner *uniformly at random*

$$J \sim \text{Uniform}(A_n(i, t_k)).$$

Let $P = [p_{\ell m}]$ be the (constrained) transition matrix. The mutation rule is a single Bernoulli trial:

$$L_{t_{k+1}}(a_i) = \begin{cases} L_{t_k}(J), & \text{with probability } p_{L_{t_k}(a_i), L_{t_k}(J)}, \\ L_{t_k}(a_i), & \text{with probability } 1 - p_{L_{t_k}(a_i), L_{t_k}(J)}. \end{cases}$$

Model Initialisation

We already defined our agent set A in a previous paragraph so now we must initial each state of our agent $a_i \in A$. We initial our model by defining both $L(a_i)$ and $P(a_i)$ for each agent $a_i \in A$.

Initial Language Distribution. We can define the initial language distribution by a probability distribution π over our language set U . Each agent $a_i \in A$ will have its language state $L(a_i)$ initialised by a random draw from the distribution π .

Initial Position Distribution. We will define our initial position for each agent $a_i \in A$ by a predefined spatial distribution over our space S . based on our applicable context. For example, we could define a uniform distribution over S or a clustered distribution based on population centers.

Parameters and User Inputs

To allow for flexibility in our model, we will define several parameters that can be adjusted by the user, we have already that the radius for defining our **interactable neighbours** r can be variable based on our users discretion. Additionally, we will define our mutation threshold p_{input} as a user-input parameter which will determine the minimum mutation probability required for interaction between two agents. The user will also be able to define the number of agents in A such that $|A|$ is a finite but definable size based on the users discretion.

Simulation Execution

With all components defined, we can now execute our simulation over a defined number of ticks t which will be at our users discretion. At each tick t_k , we will perform the following steps for each agent $a_i \in A$:

1. Determine the set of interactable neighbors A_n for agent a_i .
2. Apply the movement rule Φ to update the position $P(a_i)$.
3. Apply the mutation rule to potentially update the language state $L(a_i)$.
4. Proceed to the next tick t_{k+1} .
5. Repeat until the final tick t_n is reached.
6. Collect and analyze the results of the simulation, focusing on language distribution and mutation patterns over time.

Building the Model

To build the model, we used a python framework called Mesa, designed for agent-based modelling. We began by creating two main class components: the **LanguageAgent** class and the **LanguageModel** class. Our agent class served as the blueprint for each individual agent in our model, defining their specific properties and behaviors as mentioned in the previous section. We would include methods for movement as such,

```
def moveHelper(self):
    interactableNeighbours = self.getInteractableNeighbours()
    eps = 1e-6
    scores = {d: eps for d in self.moves}
    for neighbor in interactableNeighbours:
        direction = self.binaryMoveHelper(self.pos, neighbor.pos)
        weight = self.model.P[self.language, neighbor.language]
        scores[direction] += weight

    directions = list(scores.keys())
    weights = list(scores.values())
    if not interactableNeighbours:
        p1 = self.random.random()
        if p1 > 0.6:
            drow, dcol = self.moves[self.random.choice(list(self.moves))]
            return (self.pos[0] + drow, self.pos[1] + dcol)
        else:
            return self.pos
    else:
        chosen_dir = self.random.choices(directions, weights=weights)
        direction = self.moves[chosen_dir]
        dx, dy = direction
        return (self.pos[0] + dx, self.pos[1] + dy)
```

that would provide the basic movement functionality for each agent. We can see here that the agent follows the principles we defined earlier, moving towards interactable neighbours based on the mutation probabilities defined in our transition matrix P .

LanguageModel Class. The **LanguageModel** class served as the foundation for our general framework, in which we can define how many agents we have, the mutation and transition probabilities, as well as how we specifically create and manage new mutations. (I thought this was the most interesting part of the code so I included it here).

```
def createLanguageMutation(self, agent1, agent2):
    lang1 = agent1.language
    lang2 = agent2.language
    oldP = self.P
```

```

nOld = self.languageNum
newId = nOld
mutation_origin = np.random.choice([lang1, lang2])
newP = np.zeros((self.languageNum + 1, self.languageNum + 1))
for i in range(self.languageNum):
    for j in range(self.languageNum):
        newP[i, j] = oldP[i, j]

newId = nOld
for j in range(nOld):
    newP[newId, j] = oldP[mutation_origin, j]
    newP[j, newId] = oldP[j, mutation_origin]

newP[newId, newId] = oldP[mutation_origin, mutation_origin] * 0.9
self.P = newP
self.normalize_P_rows()
self.languageNum += 1
self.totalMutations += 1
self.updateLanguageNetwork(newId, mutation_origin)
print(newId)
return newId

```

This is the function that creates a mutated language based of the two initial languages of the interacting agents. This function is probably the thing that changed the most in our code, as the specific inheritance properties that the language possesses really shaped the way the model behaved. Ultimately, we even had to create a stabilizer functionality for the mutations as in our initia model, languages would mutate too quickly and the model would just become a mess of different languages.

User Interface and Execution Building the user interface was honestly the hardest part as Mesa’s built in framework was painfully difficult to work with. However, after a lot of trial and error, we were able to create a functional user interface that allowed us to adjust parameters such as the number of agents, mutation and transition probabilities, interaction radius, and mutation threshold.

Upon first running a working version of the model were it didnt crash after some niche case, we observed some interesting and unrealistic behaviour. Our languages were mutating extremely quickly and after only a few ticks we had about a 100 different mutations. This lead to a process of trying to figure out where in the mathematical framework, we had created some sort of instability. After some thought, we realized that after a mutation was added, the weights added to the transition matrix were too high meaning that it would cause a snowballing effect after just one mutation happened. To fix this, we added a stabilizer in the code here

```
# mutation logic changes
```

```

i = int(self.language)                                # current language index
stability = self.model.P[i, i]                        # P[ii], after normalization
base_mu = self.model.mutationProb
scale = 0.4      # what the slider controls
effectiveMu = base_mu * (1 - stability) * scale

if P[self.language, interactAgent.language] >= self.model.probThreshold:
    # if p > self.model.probThreshold:
        if self.random.random() < effectiveMu:
            newLang = self.model.createLanguageMutation(self, interactAgent)
            self.language = newLang
            interactAgent.language = newLang
        else:
            self.language = interactAgent.language

```

which basically scales down the mutation probability based on how stable the language is. This really changed the way our model worked and made it much more realistic. After this change we could observe small changes in mutation probability and threshold values and really actually see how they affected the model.

We were really happy after making this change as it made the model much more usable and made it so that we could actually interpret what was happening in the model in terms of real world situations as our model behaved in a much more realistic way.

Results

To best interpret our model, we ran three main tests with varying mutation probability (p_μ), transition threshold probability (p_t) and interaction radius (r) values to see how they affected the behaviour of agents in our space. We will also model each time tick (t) as a generation of approximately 50 years to better interpret the results in a real world context.

Our constant variables for these tests were the grid space (50x50), number of agents (500), initial red population (275) and initial blue population (225).

Test 1 : $p_\mu = 0.01$, $p_t = 0.2$, $r = 1$

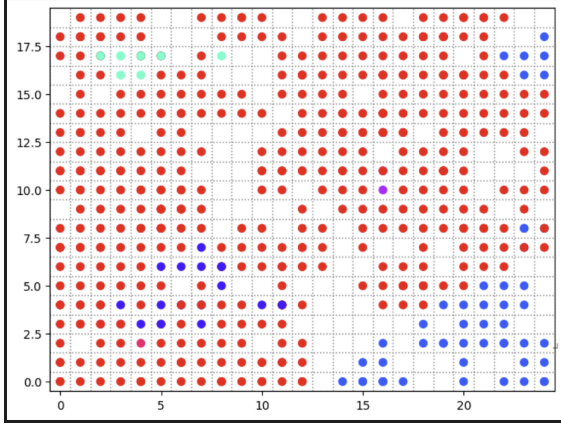


Figure 1: *
Generation 25

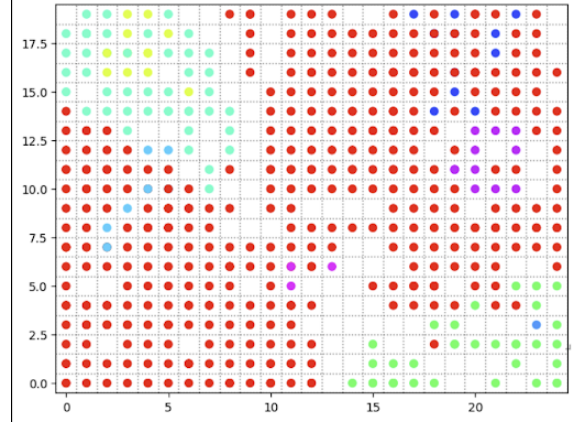


Figure 2: *
Generation 50

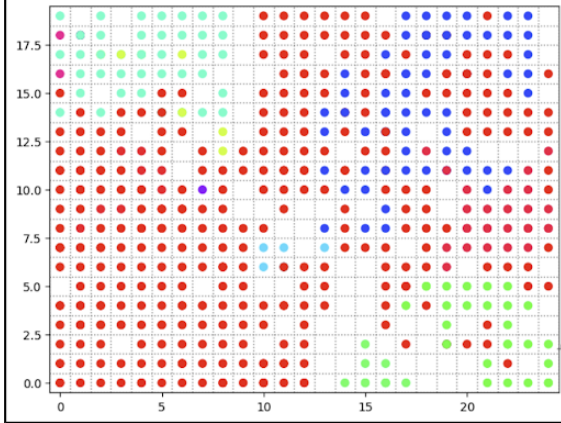


Figure 3: *
Generation 75

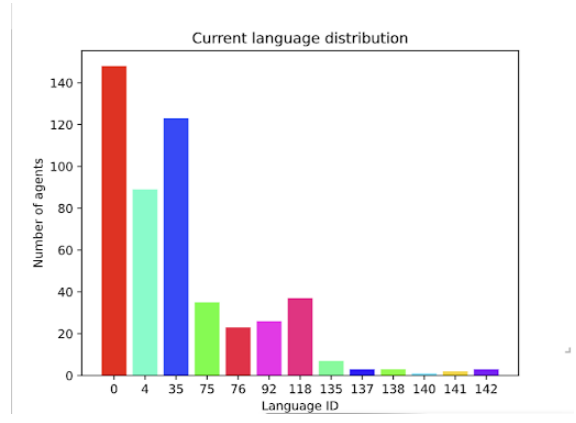


Figure 4: *
Language Distribution

During this test, we see red, the language with a slightly larger initial population, quickly become dominant over the space in the first 25 generations, leaving a small, dense community of blue speakers left. After 50 generations, the teal linguistic group grows substantially in population, and the previously blue population has adopted some features from its surrounding red community and has become green. By the 75th generation, blue has shown a resurgence and the teal and green communities have diminished. At the end of the experiment, we see these “additional” languages grow more even in footing with a ratio of 14:12:9 (red:blue:teal).

Test 2 : $p_\mu = 0.02$, $p_t = 0.3$, $r = 3$

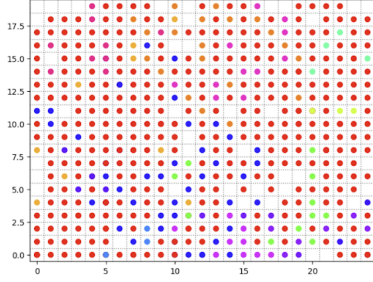


Figure 5: *
Generation 25

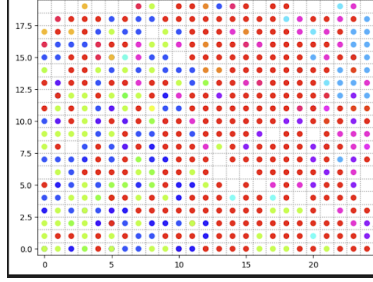


Figure 6: *
Generation 50

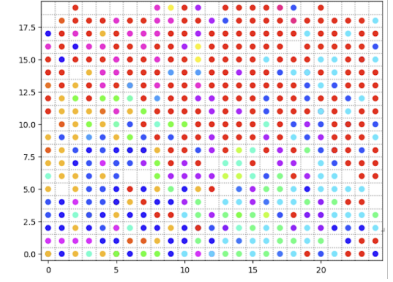


Figure 7: *
Generation 75

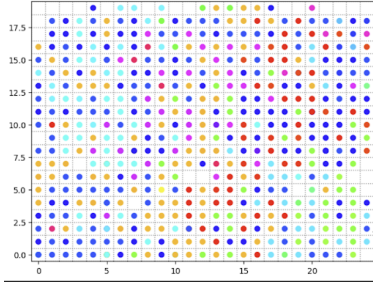


Figure 8: *
Generation 100

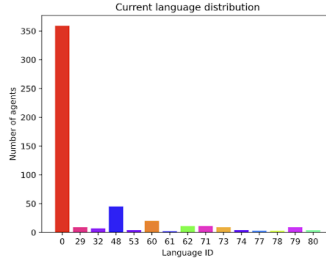


Figure 9: *
Language Distribution
(Gen 25)

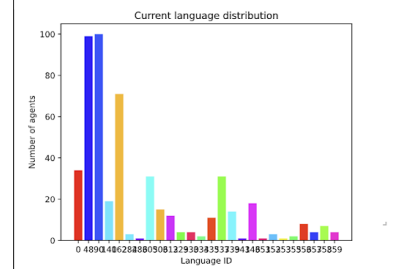


Figure 10: *
Language Distribution
(Gen 100)

After the first 25 generations of Test 2, red dominates blue, but leaves a smaller more spread out community of blue speakers with a couple green speakers. Additionally there are about 4 greenish teal speakers in a corner on their own. By generation 50, blue and green have grown and migrated to their own space on the left and the greenish teal speakers have grown in population and diverged more from red to become a dull blue. Following the 75th generation, The blue and green speakers have moved again, to the bottom half. Agents in this community have started interacting with the dull blue language speakers, and have started changing some red speakers as well into other dialects. At the end of the test, we see a large takeover by the blue and green community and lots of interactions between the green and red, resulting in a large orange spike.

Test 3 : $p_\mu = 0.005$, $p_t = 0.1$, $r = 5$

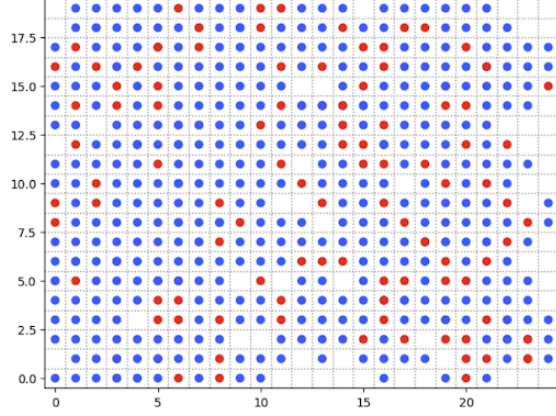


Figure 11: *
Generation 25

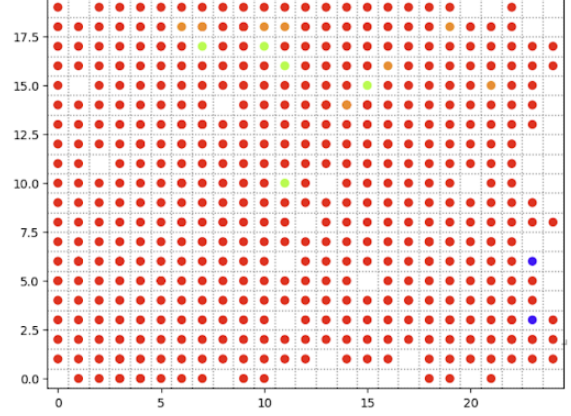


Figure 12: *
Generation 50

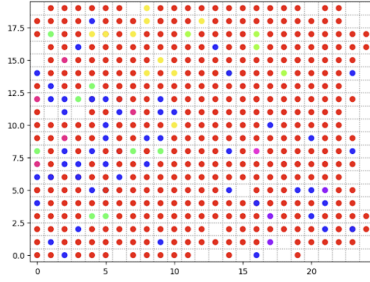


Figure 13: *
Generation 75

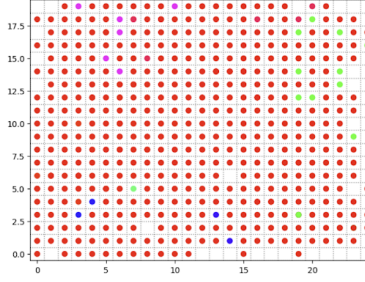


Figure 14: *
Generation 100

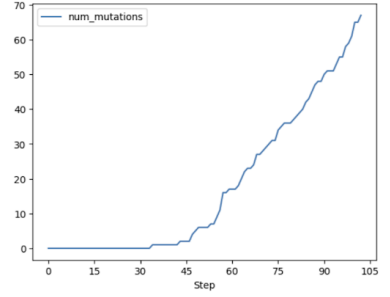


Figure 15: *
Mutation Gradient

The third test had very dramatic waves. Over the course of the first 25 generations, blue took over and reigned the grid, leaving little trace of red. Then, after 50, red took over again and left some intermediate green dialects. The small populations of blue and light green grew a little blue generation 75, and by the end of the 105th generation, red had subdued them further

We have also included Figure 15 to show how influential even the small mutation probability became as a result of how large the interaction radius was in this test. The y-axis represents the number of mutations that have occurred, and the x-axis, generations passed.

Discussion

In our model, we attempted to explore how unique languages and dialects form from the interaction, whether natural or forced, between people who speak two different languages or dialects. This is a fascinating phenomenon to study because along with the spread of languages also comes the cultural beliefs of the cultures that speak them. When languages die out, the values embedded in those languages also disappear. However, new cultural principles are also demonstrated in the formation of a new language. When one examines the world, there are many forms of similar languages, and also many unique languages all in the same areas of the world. Studying how these patterns came to be can help one understand how they might develop in the future, as well as how various cultural factors might affect and be affected by the changes. Our model also includes the individual's role in these large-scale patterns as they undergo natural mutations which are not directly related to the language that their parents speak. It has been studied that small changes in these personal language forms have an effect on the way larger language groups develop, but our model aims to show how that happens (Blythe).

When the model is run, one can see that the individual mutation probability has a noticeable effect on how many languages are formed over time. When there is a higher percentage of mutation, even if it is still a tiny one, more dialects will form in the model, and the number of speakers of each of those dialects will be fewer than when the mutation probability is much lower. This demonstrates the compounding impact of individual changes in languages, and explains that, even without interaction with other people, these mutations would lead to the development of noticeably different ways of speaking, just over a longer period of time. The other parameter that was explored was the interaction radius. When agents can only interact with people who are close to them physically, there are more likely to be smaller groups that are scattered in different places on the board. There is still one dominant language, but others can take a significant hold over groups of people. However, as the radius widens, and speakers of every language can interact with people who are farther away from them, one language will take over rapidly, and there will not be much linguistic variation. This has interesting implications for the advent of the internet, because the internet allows people to interact with people within a much larger radius than in the pre-internet times. Our model could help demonstrate how this widening of connections leads to the advent of international languages that everyone speaks.

In the future, there are many parameters of the model that could be adjusted in order to explore different conditions that are placed on languages in the real world. One place to start would be varying the number of agents that speak each language so that one language has significantly more speakers than the other. Depending on the radius and the percentage of mutation, it would be useful to study how that initial disadvantage affects both languages' chances of survival. It would be interesting to examine whether or not there is a point at which the language with fewer speakers will die out or change into another language entirely because it is outnumbered by the other language. Another aspect of the model that could be investigated is how languages change if some people, or entire language groups, have different tolerance levels for changing over to the language that they are interacting with. This would be able to mirror a power imbalance in real-life situations, where one language has more political power, and there is therefore, more impetus for the weaker language speakers

to adopt the stronger language than for the stronger language speakers to adopt the weaker language. This difference in settings could also apply to generations in which certain individual agents are much more willing to change their language compared to others, as often happens between members of different generations (Nevalainen) In these settings, one could also delve into how quickly the weaker language loses speakers, and whether or not it dies out eventually. By further studying these aspects of the model, it could provide more real-world understanding of the linguistic, and therefore cultural, phenomena that are visible in our world. While this model does not provide solutions, as it aims to explain more than to define what can be fixed, it can give a solid idea of the various factors that are affecting the languages that are spoken in the world and what languages will be spoken in the future.

Limitations and Further Work

While we were pretty happy with how our model turned out, we observe some key limitations to be considered when interpreting the results and thinking about further iterations.

First, our model assumes a simplified 2D space for agents to interact in. It does not take into account geographical features, population density variations, or social structures that can significantly impact how agents interact with each other. This is probably the biggest area for improvement where the model can be redefined. If we added a more complex set of rules for interactions based on specific types of contexts and locations, such as the aforementioned, the model would further mimic real interactions.

We also believe that the agent movement can be refined in the model to accommodate more adaptability and nuance. Optimally, we would have liked to use a machine learning-based approach for each agent. In which, agents would utilize a set of learned behaviours rather than predetermined rules in their interactions.

Finally, we would have liked to have extended our model to allow the inclusion of more initial languages and dialects in tandem with a larger agent population to accommodate this diversity. This would allow us to use our model to really observe the evolution and interactivity of real-world language above more abstract concepts.

References

- [1] Blythe, R. A., & Croft, W. (2021, June 2). How individuals change language. *PLOS One*. Retrieved from <https://pmc.ncbi.nlm.nih.gov/articles/PMC8172061/#sec002>
- [2] Byrd, A. S., Huang, Y. T., & Edwards, J. (2023, May 2). The impact of dialect differences on spoken language comprehension. *Applied Psycholinguistics*. Retrieved from <https://www.cambridge.org/core/journals/applied-psycholinguistics/article/impact-of-dialect-differences-on-spoken-language-comprehension/6305E4F885A39BBF3ED47E70C3412BBC>
- [3] Nevalainen, T., Raumolin-Brunberg, H., & Mannila, H. (2011). The diffusion of language change in real time: Progressive and conservative individuals and the time depth of change. *Language Variation and Change*, 23(1), 1–43. <https://doi.org/10.1017/S0954394510000207>